

Smart Parking – System Requirements & Engineering Characterization Document (PRD / SRS)

Written by:

Lior Cohen · Yashar Pashaiy · Dolev Halabi · Matthew Tsiplakov

Center of Projects and Industrial Cooperation

Faculty of Science, HIT – Holon Institute of Technology

Table of Contents

- Administration
- 0.1 Introduction – Existing situation (a problem or a gap definition)
- 0.2 Rationale for application development & Project Goals
- 0.3 Concepts and Definitions
- System description
- 1.1 The Character of an application
- 1.2 Functional Requirements
- 1.3 Non-Functional Requirements
- 1.4 System users
- 1.5 Usage scenarios (Use Cases) and Activity Flows
- Work Flow and Telemetry Lifecycle
- Architecture and Design
- 3.1 Client Architecture, UI Interfaces, and Dual Layout Engines
- 3.2 Server Architecture, Technological Interfaces, and REST/WebSocket Gateway
- DB Tables (Data Structure - Unified Table)
- Testing and Verification Stages
- Recommendations for Future Development
- Project Deliverables (Links)

0. Administration

0.1 Introduction – Existing situation (a problem or a gap definition)

The field of parking management today presents a severe operational and logistical challenge, especially in crowded urban centers, commercial areas, and academic campuses (such as HIT – Holon Institute of Technology). Drivers arriving at a parking lot expend substantial time and fuel aimlessly searching for an available spot, resulting in traffic bottlenecks, elevated vehicle emissions, and driver frustration.

Simultaneously, parking lot managers lack modern, centralized technological tools to obtain a real-time snapshot of facility occupancy across multi-level structures, monitor the classifications of vehicles entering (regular, disabled, dean), configure dynamic tariff policies, and supervise revenue collection. Legacy parking systems rely on mechanical ticket dispensers, magnetic cards, or human-operated exit booths that are prone to mechanical faults, slow throughput, and queue congestion at facility barriers.

The fundamental gap is the absence of an integrated, smart IoT platform that links automated vehicle tracking at facility entrances, stall-level sensor telemetry, multi-floor dynamic graphical guidance, flexible pricing calculations, and contact-free digital checkout with instant barrier clearance.

0.2 Rationale for application development & Project Goals

The SmartParking system was conceived and engineered to resolve these inefficiencies through a unified cloud, web, and IoT telemetry ecosystem.

Primary Project Goals:

- Provide sub-second real-time monitoring of parking spot occupancy and dynamic visual driver guidance without page refreshes.
- Offer an intuitive, interactive dual-layout visual interface: a procedural dynamic grid layout and high-fidelity aerial image layouts with sub-meter coordinate precision.
- Enable automated geolocation discovery via GPS and direct 1-click turn-by-turn routing through the Waze navigation platform.
- Implement a robust self-service digital billing engine featuring automated license plate lookup, tiered tariff calculations (free hours, per-minute billing, daily ceilings), PayPal Orders v2 integration, and a calibrated 15-minute exit grace period.
- Empower municipal and campus administrators with a secure, role-based dashboard for multi-level facility management, dynamic level creation/deletion, spot status override, vehicle allowlist management, and real-time session auditing.

0.3 Concepts and Definitions

- **LoRa Simulator:** A high-throughput telemetry simulation engine emulating Long Range low-power IoT parking sensors by generating and streaming batch JSON payloads to dedicated server ingestion routes.
- **IoT Sensor Node:** An edge device (simulated) positioned per parking stall detecting vehicle presence and transmitting binary state changes: free, occupied, or block.
- **Parking Session:** A persistent database audit record documenting a vehicle's license plate, entry timestamp, associated lot and stall, billing status (Payable, paid, pass), and cryptographic checkout tokens.
- **Exit Grace Period (15 Minutes):** A calibrated time window post-payment during which a driver can exit through the barrier without incurring additional parking fees.
- **Authorized Vehicle Allowlist (pass):** Vehicles registered under protected categories (e.g., Dean, Disabled) exempt from standard parking tariffs.

- **Image-Based Map Layout:** High-fidelity architectural layouts mapped to photographic aerial views with defined XY percentage coordinates, stall bounding boxes, and rotation angles.
-

1. System description

1.1 The Character of an application

- **Frontend Architecture:** A responsive Single Page Application (SPA) developed in React 19 and Vite, styled with TailwindCSS, supporting full Right-to-Left (RTL) Hebrew localization, responsive layouts, and inclusive accessibility themes.
- **Backend Processing Engine:** A Node.js server running on Express, architected around modular Service-Repository layers, centralized error handling, and IP-based rate limiting.
- **Telemetry & Real-Time Gateway:** Socket.io WebSocket server maintaining persistent bidirectional channels for instant client re-rendering upon stall state or session updates.
- **Financial Processing Pipeline:** Fully integrated with PayPal Orders API v2, managing order initialization, modal checkout, server-side capture, and asynchronous webhook validation.
- **Transactional Receipt Engine:** Automated HTML receipt generation with secure HTTPS delivery through a Google Apps Script web app, authenticated using a server-side shared secret.
- **Persistent Data Layer:** MongoDB NoSQL database utilizing Mongoose schemas with optimistic concurrency control, indexed query optimizations, and field-level data hiding.

1.2 Functional Requirements

Driver Facing Capabilities:

- **Municipal & Geolocation Discovery:** The system shall allow drivers to search for parking facilities by municipality name with auto-complete suggestions or by utilizing GPS geolocation to retrieve facilities within an expanding radius.
- **Proximity & Waze Integration:** The system shall display computed driving distances in meters or kilometers and provide a direct link to launch turn-by-turn navigation in Waze via facility GPS coordinates.

- **Interactive Visual Guidance:** The system shall render multi-floor parking layouts showing stall availability, stall numbers and asphalt classification badges (disabled, dean).
- **Pricing Transparency:** The system shall provide drivers with instant hover tooltips and interactive modal dialogs displaying detailed tariff terms (free initial hours, minute rates, maximum daily ceilings, flat fees).
- **Personal Favorites & History:** The system shall persist driver favorite facilities and recent city searches across sessions using local storage.

Billing, Payment & Checkout Capabilities:

- **Automated Plate Lookup:** The system shall allow drivers to query outstanding parking sessions by entering an alphanumeric vehicle license plate.
- **Dynamic Fee Computation:** The system shall calculate parking charges dynamically using the facility's piecewise tariff structure, deducting completed payments and exempting allowlisted vehicles.
- **Digital Payment Gateway:** The system shall integrate with a payment method to create, authorize, and capture payments securely using minor integer currency units (ILS agorot).
- **Exit Grace Period:** The system shall activate a 15-minute exit countdown timer upon payment completion, permitting gate departure without additional fees.
- **Automated Digital Receipting:** The system shall accept an email address upon payment completion and dispatch a branded HTML receipt containing transaction details via mail service.

IoT Telemetry & Sensor Simulation Capabilities:

- **Batch Telemetry Ingestion:** The system shall provide an HTTP POST endpoint (/lora/update-spot) accepting batched JSON messages containing stall state changes and session commands.
- **Automated Session Management:** Ingested vehicle arrival messages shall automatically initialize a database ParkingSession document and transition the corresponding ParkingSpot to occupied.

Administrative & Facility Management Capabilities:

- **Role-Based Access Control:** The system shall restrict manager access to authorized municipal territories (authorizedCities) authenticated via JWT cookies.
- **Facility CRUD:** The system shall allow administrators to create, update, and delete parking facilities, including address, GPS coordinates, floor counts, and pricing configurations.
- **Dynamic Level Management:** The system shall enable administrators to dynamically add new parking floors or delete existing levels, automatically recalculating total spot capacities.
- **Spot Maintenance Override:** The system shall permit administrators to manually toggle individual stalls to blocked for maintenance and adjust spot privilege categories (disabled, dean).
- **Authorized Vehicle Allowlists:** The system shall provide an administrative interface to register, view, and remove exempt license plates with automated character normalization.
- **Real-Time Session Auditing:** The system shall provide a modal displaying all currently parked vehicles, license plates, assigned stalls, and ingress timestamps.

Accessibility & System Customization Capabilities:

- **Colorblind Accessibility:** The system shall provide a dedicated Colorblind Mode substituting standard red/green visual cues with high-contrast shapes, icons, and text badges.
- **Night Driving Dark Mode:** The system shall offer a toggleable Dark Theme complying with WCAG contrast standards.
- **Large Text Mode:** The system shall support a large text accessibility mode scaling typographical components across all views.
- **Theme Customization:** The system shall support multiple color palettes (Classic, Ocean, Earth).
- **Cookie Consent:** The system shall display a persistent cookie consent banner to comply with digital privacy standards.

1.3 Non-Functional Requirements

- **Performance & Latency:** WebSocket telemetry latency under 500ms. REST API responses under 200ms. Vector graphics and lightweight CSS ensure 60 FPS rendering on mobile devices.
- **Security & Authentication:** Passwords hashed with Bcrypt (salt factor 10). Admin authentication secured with JWT stored in HttpOnly, SameSite cookies. Sensitive endpoints protected by IP rate limiters (express-rate-limit).
- **Financial Compliance:** Zero storage of sensitive credit card numbers; cardholder data is securely handled by PayPal PCI-DSS compliant infrastructure.
- **Data Privacy & GDPR:** License plate allowlists and token hashes are hidden from public queries. User consent is managed via a persistent Cookie Consent Banner.
- **Scalability:** Stateless Express backend and indexed MongoDB collections support horizontal scaling across dozens of cities and hundreds of parking facilities.

1.4 System users

1. System Admin / Manager:

- Holds management privileges scoped to assigned municipal jurisdictions (authorizedCities).
- Manages parking facilities: creating, editing, and deleting parking lots, configuring GPS coordinates, and setting tiered pricing policies.
- Manages facility levels: dynamically adding or deleting levels and adjusting spot distribution.
- Manages individual parking spots: blocking spots for maintenance and assigning spot classifications (regular, disabled, dean).
- Maintains the Authorized Vehicles Allowlist with automated license plate normalization.
- Monitors real-time graphs, occupancy metrics, and currently parked vehicles.

2. Regular Driver:

- Discovers facilities via municipal search or GPS geolocation ("Find Near Me") with dynamic search radius expansion.
- Views real-time multi-level occupancy on procedural or photographic aerial maps.
- Inspects detailed tariff schedules via hover tooltips and pricing modals.
- Launches 1-click turn-by-turn routing via Waze.
- Saves favorite parking lots and accesses recent search history.
- Conducts self-service checkout by license plate via PayPal, monitors the 15-minute exit timer, and receives email invoices.

1.5 Usage scenarios (Use Cases) and Activity Flows

1.5.1 Use Case: Driver Parking Search, Geolocation Discovery, and Waze Navigation

- **Actor:** Regular Driver
- **Goal:** Locate an optimal, available parking spot and navigate directly to it.
- **Activity Flow:**
 1. The driver visits the web app and either selects a city or clicks "Find Near Me".
 2. The system queries browser geolocation (navigator.geolocation) and calls the backend /api/parking/nearby endpoint.
 3. The backend calculates distances and returns nearby lots sorted by proximity.
 4. The driver inspects the lot card to see total spots, currently available spots, and pricing information.
 5. The driver clicks the Waze navigation icon to launch direct GPS navigation to the facility coordinates.
 6. Upon arrival, the driver opens the facility view, selects floor levels, and views open spots in real time.

1.5.2 Use Case: Real-Time Spot Occupancy & Session Tracking (LoRa Simulator)

- **Actor:** LoRa Simulator / Server
- **Goal:** Ingest sensor telemetry and broadcast real-time status changes across all clients.
- **Activity Flow:**
 1. The LoRa Simulator generates a batch JSON payload containing stall status changes (free, occupied, block) and session actions (create, remove).
 2. The payload is sent via HTTP POST to /lora/update-spot.
 3. The server updates the ParkingSpot in the database and creates/updates a ParkingSession.
 4. The server emits a parking-spot-updated event to public city rooms and a parking-session-updated event to authorized admin channels via Socket.io.
 5. Connected client browsers receive the event and re-render the stall visually (e.g., displaying animated parked car sprites or green free indicators) instantly.

1.5.3 Use Case: Driver Self-Service Billing, PayPal Payment, and Exit Grace Timer

- **Actor:** Regular Driver
- **Goal:** Settle parking fees digitally and exit the facility safely within the grace period.
- **Activity Flow:**
 1. The driver navigates to /payment and enters their vehicle license plate number.
 2. The server queries active sessions, verifies entry time, checks exempt allow list, and calculates the fee according to the lot's pricing rules.
 3. If the vehicle is exempt or within the initial free hours, the screen displays "No Payment Required" with an active exit pass.
 4. If payment is due, an itemized invoice is shown (entry time, elapsed duration, applied tariff, and net amount in ILS).
 5. The driver clicks to pay; the system initializes a PayPal order via /api/paypal/orders using cryptographic checkout credentials (checkout ID and checkout token).
 6. The driver approves the order in the PayPal modal; the backend captures payment via /api/paypal/orders/:id/capture and records a Parking Payment document.
 7. After successful payment, the system activates a 15-minute exit grace period. The driver must leave the parking lot within this window; if the vehicle remains after the grace period expires, parking charges begin accumulating again based on the lot's pricing rules, minus amounts already paid.
 8. The driver may enter an email address to receive an automated HTML receipt. The backend validates the completed payment, generates the receipt from trusted server-side payment data, and sends it securely over HTTPS through a Google Apps Script web app authenticated with a server-side shared secret.

1.5.4 Use Case: Manager Dashboard and Multi-Level Facility Management

- **Actor:** System Admin / Manager
- **Goal:** Monitor occupancy, adjust infrastructure, and configure tariffs.
- **Activity Flow:**
 1. Manager logs into /login with encrypted credentials and receives a secure JWT cookie.
 2. The Dashboard displays facilities filtered by the manager's assigned cities.
 3. The manager can add a new parking facility, specifying address, GPS coordinates, levels count, spot distribution, and pricing schedules (free hours, per-minute rate, daily cap).
 4. The manager can dynamically add or delete levels and individual parking spots.
 5. The manager can open the Authorized Vehicles Modal to add or remove privileged license plates.
 6. The manager can click "View Parked Cars" to view active parking sessions and entry timestamps in real time.

2. Work Flow and Telemetry Lifecycle

1. **Hardware Simulation:** The LoRa simulator continuously generates batch telemetry representing vehicle arrivals and departures.
2. **Data Transfer & Ingestion:** JSON payloads stream via HTTP POST to the central Express backend.
3. **Backend Processing & Business Logic:**
 - Updates stall status (free, occupied, block).
 - Initializes or updates Parking session records, generating unique checkout ID and SHA-256 token hashes.
 - Checks vehicle allow list to tag exempt vehicles with checkout status: 'pass'.
4. **Real-Time Event Broadcasting:** Socket.io emits scoped updates to connected drivers and managers.
5. **Dynamic Client Rendering:** React components re-render immediately, updating counters, car models, and occupancy badges.
6. **Checkout & Financial Settlement:** Drivers settle balances via PayPal; backend captures transactions, logs ParkingPayment records, triggers the 15-minute exit grace window, and dispatches email receipts.
7. **Departure & Release:** When the vehicle exits, sensor telemetry signals stall release, returning the stall status to free.

3. Architecture and Design

3.1 Client Architecture, UI Interfaces, and Dual Layout Engines

The client is built in React 19 with modular Context Providers:

- **AuthContext:** Manages admin authentication state, login, and session validation.
- **SocketContext:** Maintains the persistent WebSocket connection and event handlers.
- **SettingsContext:** Persists UI preferences (Dark Mode, Colorblind Mode, Large Text, Theme Color) in localStorage.

Key Client Pages & UI Interfaces:

- **HomePage (/):** City search, "Find Near Me" button, recent searches, favorites carousel, and parking cards.
- **ParkingLotView:** Interactive floor-by-floor view with level navigation and spot click actions.
- **Dashboard (/dashboard):** Administrative portal for municipal selection, lot CRUD, level management, and car audits.
- **PaymentPage (/payment):** Multi-step checkout wizard (Plate search, Invoice summary, PayPal button, 15-minute countdown, Receipt email form).
- **PriceListPage (/price-list):** Public breakdown of pricing policies across all registered facilities.
- **SettingsPage (/settings):** Accessibility controls (Colorblind, Dark Mode, Large Text, Color Theme).

Dual Layout Engine Implementation:

1. Dynamic Procedural Layout (DynamicParkingLayout.jsx):

- Procedurally generates symmetrical 8-slot rows (up to 32 stalls per level).
- Renders realistic asphalt roads with painted directional traffic arrows, yellow dashed center lines, wheel stoppers, and painted asphalt markings for disabled and dean stalls.

2. Realistic Image-Based Layout (ParkingMapLayout.jsx):

- Uses high-resolution aerial photography for real-world parking lots (e.g., Mediatheque Holon, Haifa Port, Beersheba Center).
- Maps individual parking stalls using normalized percentage coordinates (x, y, width, height, and rotation).

3.2 Server Architecture, Technological Interfaces, and REST/WebSocket Gateway

The backend is a Node.js and Express application backed by MongoDB through Mongoose. It follows a pragmatic layered Service–Repository architecture: route handlers expose the application interfaces, services implement business logic, and repositories encapsulate most database operations. Socket.IO is attached to the same HTTP server to provide real-time parking updates.

- **REST and Gateway Layer:**

- **/health:** Public lightweight server-availability endpoint used for external monitoring.
- **/api/auth:** Manager registration, login, logout, current-user verification, and CSRF-token issuance. Authentication uses a JWT stored in an HTTP-only cookie.
- **/api/parking:** Public city, parking-lot, pricing, nearby-location, lot-detail, payment-session lookup, and receipt-email endpoints. It also contains authenticated management endpoints for parking lots, levels, spaces, and authorized vehicles.
- **/api/paypal:** PayPal order creation, payment capture, and verified webhook processing.
- **/lora:** API-key-protected parking-space telemetry ingestion and simulator updates.
- **/api/db:** Separately API-key-protected database initialization and mock-data seeding operations.

- **Services Layer:**
 - **authService:** Authentication and manager-account operations.
 - **parkingService:** Parking-lot, level, space, authorization, and transactional management operations.
 - **parkingSessionService:** Active-session lookup, payment eligibility, pricing, exemptions, and exit-grace-period handling.
 - **loraServices:** Processes IoT parking-space updates and creates or updates the corresponding parking sessions.
 - **paypalApiService:** Low-level communication with the PayPal API.
 - **paypalCheckoutService:** Secure checkout credential validation, order creation, capture, payment persistence, and webhook processing.
 - **parkingReceiptService:** Validates completed payments and generates receipt content from trusted server-side payment data.
 - **mailService:** Sends receipt HTML and text to the configured Google Apps Script email service through HTTPS.
 - **socketService:** Manages Socket.IO connections, room subscriptions, authentication, and real-time event delivery.
- **Repository and Data-Access Layer:**
 - **cityRepo**
 - **parkingLotRepo**
 - **parkingSpotRepo**
 - **parkingSessionRepo**
 - **parkingPaymentRepo**
 - **paypalWebhookEventRepo**
 - **userRepo**

These repositories encapsulate MongoDB queries for the corresponding Mongoose models.

- **Middleware and Security Layer:**

- **protect:** Validates the JWT authentication cookie and loads the authenticated manager.
- **requireCsrf:** Protects cookie-authenticated state-changing requests using CSRF-token validation.
- **createRateLimiter:** Provides MongoDB-backed, shared rate limiting using request-specific keys such as IP addresses, checkout identifiers, and PayPal order identifiers.
- **requireLoraApiKey:** Protects LoRa ingestion routes using the **LORA_API_KEY**.
- **requireSetupApiKey:** Protects database setup and seed routes using the **DB_SETUP_API_KEY**.
- **CORS policy:** Restricts HTTP and Socket.IO access to configured client origins while supporting credentialed cookie requests.

- **Real-Time WebSocket Interface:**

- Socket.IO distributes live parking-space and parking-session changes.
- Clients subscribe to parking updates through city-based rooms.
- Public updates contain sanitized parking-space information and do not expose license plate numbers.
- Detailed session information is emitted only to authenticated managers authorized for the relevant city.
- The Socket.IO gateway uses the same configured origin policy as the REST API.

4. DB Tables (Data Structure - Unified Table)

Table / Object Name	Key Fields & Data Types	Field Descriptions & Constraints	Relationships
City	<code>_id</code> (ObjectId), <code>name</code> (String), <code>authorizedUsers</code> (Array of ObjectId), <code>parkingLots</code> (Array of ObjectId)	<code>name</code> is required and unique (e.g., Holon, Haifa). Holds lists of authorized admins and assigned lots.	Belongs to municipal administration. Referenced by <code>ParkingLot</code> and <code>User</code> .
ParkingLot	<code>_id</code> (ObjectId), <code>name</code> (String), <code>city</code> (ObjectId), <code>address</code> (String), <code>totalSpots</code> (Number), <code>levels</code> (Number), <code>location</code> (Subdocument: <code>lat</code> , <code>lng</code>), <code>pricing</code> (Subdocument: <code>isFree</code> , <code>freeFirstHours</code> , <code>pricePerMinute</code> , <code>fullDayPriceMinor</code> , <code>parkingFeeMinor</code>), <code>currency</code> (String: ILS), <code>authorizedVehicles</code> (Array of Strings)	Manages parking lot metadata, GPS coordinates for Waze, vertical floor count, tiered pricing schedule, and hidden allowlist.	Belongs to a City (N:1). Associated with <code>ParkingSpots</code> , <code>Sessions</code> , <code>Payments</code> , and <code>Cameras</code> .

ParkingSpot	_id (ObjectId), parkingLot (ObjectId), level (Number), spotNumber (String), status (Enum: free, occupied, block), type (Enum: regular, disabled, dean), currentCarLicensePlate (String)	Manages physical parking stall status, floor level, spot number identifier, and current parked car license plate.	Belongs to a ParkingLot (N:1). Optionally referenced by ParkingSession.
ParkingSession	_id (ObjectId), carLicensePlate (String), parkingLot (ObjectId), parkingSpot (ObjectId), entryTime (Date), checkoutId (String), checkoutTokenHash (String), checkoutExpiresAt (Date), checkoutStatus (Enum: Payable, paid, pass), checkoutPaymentStarted (Boolean), checkoutConsumedAt (Date)	Tracks active vehicle stays, entry timestamps, cryptographic checkout tokens (SHA-256 hidden), and payment state (Payable, paid, or pass).	Linked to a ParkingLot and ParkingSpot. Referenced by ParkingPayment.
ParkingPayment	_id (ObjectId), parkingSession (ObjectId), parkingLot (ObjectId), checkoutId (String), amountMinor (Number), currency (String: ILS), paymentProvider (String: paypal), paypalOrderId (String), paypalCaptureId (String), paypalPaymentStatus (Enum), paidAt (Date), fulfilledAt (Date)	Immutable financial audit record storing captured transaction IDs, integer amounts in minor units (agorot), and payment timestamps.	Belongs to a ParkingSession (N:1) and ParkingLot (N:1).

PayPalWebhookEvent	_id (ObjectId), paypalEventId (String), eventType (String), processingStatus (Enum: PROCESSING, PROCESSED, FAILED), processedAt (Date), lockUntil (Date)	Idempotency log guaranteeing external PayPal webhook notifications are processed exactly once without race conditions.	Standalone external integration audit table.
Camera	_id (ObjectId), parkingLot (ObjectId), cameraType (Enum: entry, exit), ipAddress (String)	Physical or simulated ALPR cameras capturing vehicles entering and exiting the parking facility.	Belongs to a ParkingLot (N:1).
User	_id (ObjectId), fullName (String), email (String), password (String), authorizedCities (Array of ObjectId)	Stores administrator credentials, Bcrypt-encrypted passwords, and assigned municipal management privileges.	References City (M:N relationship via authorizedCities).

5. Testing and Verification Stages

1. Automated Unit Testing (node:test & node:assert/strict):

- Automated verification of pricing calculations across edge cases:
 - Free parking lots return zero charge regardless of stay duration.
 - Lots with zero free hours charge immediately from minute one.
 - Free initial hours apply before minute billing starts.
 - Multi-day stays charge full-day blocks plus remainder minutes.
 - Fractional elapsed minutes round safely to integer minor units.
 - Legacy flat-fee lots maintain backwards compatibility.

2. Financial Sandbox & Webhook Verification:

- End-to-end PayPal Sandbox order creation and server-side capture verification.
- Testing idempotent handling of asynchronous webhooks to prevent duplicate charges.
- Testing 15-minute exit grace period expiration and debt recalculation.

3. Security & Rate Limiting Verification:

- Stress-testing /api/parking/session/lookup and /api/paypal/orders to confirm 429 Too Many Requests responses when thresholds are exceeded.
- Verifying that sensitive fields (password, checkoutTokenHash, authorizedVehicles) are excluded from public REST responses.

4. Cross-Browser & Accessibility Testing:

- Verifying responsive rendering across Desktop, Tablet, and Mobile devices.
- Testing UI contrast and readability under Colorblind Mode, Dark Mode, and Large Text Mode.

6. Recommendations for Future Development

Note: Billing, PayPal Integration, Dynamic Pricing, Exit Grace Timers, and Email Receipts were originally planned for future phases, but have now been fully developed, tested, and integrated into the live production system.

Recommended future research and development includes:

1. Predictive Occupancy Engine (Machine Learning):

- Training time-series forecasting models on historical ParkingSession records to predict peak occupancy hours and recommend optimal arrival times to drivers.

2. Native Mobile App with Bluetooth Low Energy (BLE) / NFC:

- Developing a native mobile application (React Native / Flutter) utilizing BLE beacons or NFC to automatically trigger gate barrier opening upon vehicle approach.

3. Physical Edge ALPR Camera Deployment:

- Deploying physical edge camera microcomputers (Raspberry Pi / Nvidia Jetson) running OpenCV and YOLO-based automatic number-plate recognition (ALPR) streaming RTSP video directly into the /lora session ingestion pipeline.

4. Algorithmic Dynamic Congestion Pricing:

- Introducing demand-responsive pricing algorithms that adjust minute rates dynamically based on real-time occupancy to balance parking demand across campus facilities.

7. Project Deliverables (Links)

- **Characterization & Engineering Specification (PRD / SRS):** This document.
- **Presentation File:** Attached in the final project distribution package.
- **Project Poster:** Visual summary poster attached in the final project package.
- **Demonstration Video:**
https://youtu.be/e0t_uYC5PUI?si=2aZB5f7lfrlZ07cA
- **Source Code Repository (GitHub):**
<https://github.com/Drakomt/SmartParking>